Chapitre 4 : Templates et réactivité

> **Objectifs** : maîtriser la syntaxe des templates et les directives de base, comprendre `ref` et `reactive`, utiliser les propriétés calculées et les watchers. *Tous les exemples utilisent* `<script setup>`.

> **Comment lire les exemples** : dans le code Vue, les commentaires `<!-- → ... -->` (dans le template) et `// →` (dans le script) indiquent **ce qui s'affiche ou la valeur obtenue**. Les autres commentaires expliquent la ligne. Chaque exemple peut être collé dans un composant `.vue` d'un projet `create-vue` pour être testé.

---

1. Interpolation et expressions

L'**interpolation** `{{ ... }}` insère une valeur JavaScript dans le texte de la page. Elle se met à jour automatiquement quand la valeur change.

```
<script setup>
import { ref } from "vue";

const prenom = ref("Zeineb");   // donnée réactive de type texte
const age = ref(21);          // donnée réactive de type nombre
</script>

<template>
  <!-- Une variable : dans le template, pas besoin de .value -->
  <p>Bonjour {{ prenom }}</p>
  <!-- → Bonjour Zeineb -->

  <!-- Un calcul -->
  <p>Dans 5 ans : {{ age + 5 }} ans</p>
  <!-- → Dans 5 ans : 26 ans -->

  <!-- Un ternaire : choisit une valeur selon une condition -->
  <p>{{ age >= 18 ? "Majeur" : "Mineur" }}</p>
  <!-- → Majeur -->

  <!-- Un appel de méthode sur la valeur -->
  <p>{{ prenom.toUpperCase() }}</p>
  <!-- → ZEINEB -->
</template>
```

Règles :

- `{{ }}` accepte **une expression JavaScript** (qui produit une valeur) : variable, calcul, appel de fonction, ternaire. Pas d'**instruction** : pas de `if`, `for`, ni de déclaration `let` ;
- le contenu est **échappé** (sécurité contre XSS) : si la valeur contient du HTML, il s'affiche comme du texte.

```
<script setup>
const html = "<b>gras</b>";
</script>

<template>
  <p>{{ html }}</p>
  <!-- → affiche littéralement : <b>gras</b>   (les balises ne sont PAS
interprétées) -->

  <p v-html="html"></p>
  <!-- → affiche : gras  (en caractères gras : le HTML est interprété) -->
</template>
```

> ⚠ `v-html` insère du HTML brut : **dangereux** avec un contenu saisi par un utilisateur (injection de script, XSS). À réserver à un contenu de confiance.

```
<!-- ❌ Ceci ne fonctionne PAS : une instruction n'est pas une expression -->
<p>{{ if (age > 18) { "majeur" } }}</p>
<!-- → erreur de compilation -->
<!-- ✅ Utiliser le ternaire -->
<p>{{ age > 18 ? "majeur" : "mineur" }}</p>
```

---

2. Réactivité : `ref` et `reactive`

2.1 `ref` : valeur réactive

`ref(valeur)` crée une donnée **réactive** : quand elle change, tout ce qui l'utilise (template, `computed`, `watch`) est mis à jour.

```
import { ref } from "vue";

const compteur = ref(0);
console.log(compteur);          // → RefImpl { value: 0 }   (un objet qui enveloppe
la valeur)
console.log(compteur.value);    // → 0
compteur.value++;               // dans <script> : .value est OBLIGATOIRE pour
lire/écrire
console.log(compteur.value);    // → 1
```

Dans le `<template>`, `.value` est **inutile** : Vue le « dépaquette » automatiquement.

```
<script setup>
import { ref } from "vue";
const compteur = ref(0);
```

```
</script>

<template>
  <p>{{ compteur }}</p>                    <!-- ✅ → 0  (pas de .value dans le
template) -->
  <button @click="compteur++">+1</button>   <!-- ✅ dans un template aussi, pas
de .value -->
</template>
```

`ref` fonctionne avec tous les types : nombre, chaîne, booléen, tableau, objet.

```
const utilisateur = ref({ nom: "Ali", age: 20 });

utilisateur.value.age++;        // modification d'une propriété : réactif en
profondeur
console.log(utilisateur.value.age);   // → 21

utilisateur.value = { nom: "Zeineb", age: 22 };   // remplacement complet de
l'objet : réactif aussi
console.log(utilisateur.value.nom);   // → "Zeineb"
```

2.2 `reactive` : objet réactif

`reactive(objet)` rend un **objet** (ou un tableau) réactif **sans** enveloppe `.value`.

```
import { reactive } from "vue";

const etat = reactive({ nom: "Ali", age: 20, tags: ["vue"] });

etat.age++;              // pas de .value
etat.tags.push("js");
console.log(etat.age);   // → 21
console.log(etat.tags);  // → ["vue", "js"]
```

Limites de `reactive` :

```
// 1) Ne fonctionne qu'avec des objets / tableaux, pas avec des primitifs
const n = reactive(0);
// → avertissement Vue : "value cannot be made reactive: 0"  (n n'est pas réactif)

// 2) On ne peut pas remplacer l'objet entier
let o = reactive({ a: 1 });
o = { a: 2 };
// → la nouvelle valeur est un objet ORDINAIRE : la réactivité est perdue
//   (avec const, JavaScript refuse même l'affectation : TypeError)

// 3) La déstructuration copie les valeurs : on perd la réactivité
```

```
const e = reactive({ nom: "Ali" });
const { nom } = e;
e.nom = "Zeineb";
console.log(nom);        // → "Ali"   (nom est une simple copie, plus liée à e)
console.log(e.nom);      // → "Zeineb"
```

Pour déstructurer en gardant la réactivité : `toRefs`.

```
import { reactive, toRefs } from "vue";

const etat2 = reactive({ nom: "Ali", age: 20 });
const { nom: n1, age } = toRefs(etat2);   // chaque propriété devient un ref lié à etat2

etat2.nom = "Zeineb";
console.log(n1.value);   // → "Zeineb"   (reste synchronisé grâce à toRefs)
```

2.3 Que choisir ?

	`ref`	`reactive`
Primitifs (nombre, texte, booléen)	✅	❌
Objets / tableaux	✅	✅
Remplacement complet de la valeur	✅ (`x.value = …`)	❌
Accès dans le script	`.value`	direct
Déstructuration	Possible sur le contenu de `.value`	Perd la réactivité

> **Recommandation du cours : utiliser `ref` par défaut**, partout. Un seul modèle mental, aucune perte de réactivité surprenante. `reactive` est acceptable pour un objet « état » local qui ne sera jamais remplacé (par exemple un formulaire).

2.4 Mutation des tableaux et objets

Vue 3 détecte l'ajout et la suppression de propriétés, ainsi que les modifications de tableaux :

```
const liste = ref([1, 2, 3]);

liste.value.push(4);                 // ✅ réactif
console.log(liste.value);            // → [1, 2, 3, 4]

liste.value[0] = 99;                 // ✅ réactif (en Vue 3, l'accès par indice est détecté)
console.log(liste.value);            // → [99, 2, 3, 4]
```

```
liste.value = liste.value.filter(n => n > 3);   // ✅ remplacement du tableau
console.log(liste.value);                       // → [99, 4]
```

```
const personne = ref({ nom: "Ali" });
personne.value.ville = "Tunis";      // ✅ ajout d'une propriété : détecté en Vue 3
delete personne.value.nom;           // ✅ suppression : détectée aussi
console.log(personne.value);         // → { ville: "Tunis" }
```

---

3. Directives

Une **directive** est un attribut spécial préfixé par `v-`. Elle dit à Vue **comment lier** l'état au DOM.

3.1 `v-bind` (`:`) : lier un attribut

`v-bind` remplace la valeur **fixe** d'un attribut HTML par une **expression** qui suit l'état.

```
<script setup>
import { ref } from "vue";

const url = ref("https://vuejs.org");
const desactive = ref(true);
const nom = ref("logo");   // sert à construire un chemin d'image
</script>

<template>
  <!-- Forme complète -->
  <a v-bind:href="url">Vue</a>
  <!-- → <a href="https://vuejs.org">Vue</a> -->

  <!-- Raccourci « : » (le plus utilisé) -->
  <a :href="url">Vue</a>
  <!-- → même résultat -->

  <!-- Attribut booléen : ajouté si la valeur est vraie, retiré sinon -->
  <button :disabled="desactive">Envoyer</button>
  <!-- → <button disabled>Envoyer</button>   (desactive vaut true). Avec false : le mot « disabled » disparaît -->

  <!-- Valeur construite avec un template literal -->
  <img :src="`/images/${nom}.png`" :alt="nom">
  <!-- → <img src="/images/logo.png" alt="logo"> -->

  <!-- Sans « : », l'attribut est un texte fixe -->
  <a href="url">Faux</a>
  <!-- → <a href="url">  (le texte littéral « url », pas la variable !) -->
</template>
```

Syntaxe abrégée de même nom : `<input :value>` équivaut à `<input :value="value">` (Vue ≥ 3.4).

3.2 `v-on` (`@`) : écouter un événement

`v-on` associe un **gestionnaire** (une fonction ou une expression) à un événement DOM.

```
<script setup>
import { ref } from "vue";

const compteur = ref(0);

// Une méthode ordinaire : reçoit l'événement natif en premier paramètre
function incrementer(event) {
  compteur.value++;
  console.log(event.type);     // → "click"
  console.log(event.target);   // → <button>…</button>  (l'élément cliqué)
}

// Une méthode avec un paramètre fourni par le template
function ajouter(n) {
  compteur.value += n;
}
</script>

<template>
  <!-- Forme complète / raccourci « @ » -->
  <button v-on:click="compteur++">+1 (forme complète)</button>
  <button @click="compteur++">+1 (raccourci)</button>
  <!-- → à chaque clic : compteur augmente de 1 -->

  <!-- Nom de méthode : appelée avec l'événement -->
  <button @click="incrementer">+1 via méthode</button>

  <!-- Appel avec argument : écrire les parenthèses -->
  <button @click="ajouter(5)">+5</button>
  <!-- → à chaque clic : compteur augmente de 5 -->

  <!-- Argument ET événement natif : variable spéciale $event -->
  <button @click="ajouter(5, $event)">+5 avec l'événement</button>

  <p>Compteur : {{ compteur }}</p>
  <!-- → après un clic sur « +1 » puis « +5 » : Compteur : 6 -->
</template>
```

> `@click="incrementer"` (sans parenthèses) **passe la fonction** à Vue, qui l'appellera au clic.
> `@click="incrementer()"` écrit une **expression** que Vue exécutera au clic : même effet ici, mais l'événement n'est alors pas transmis automatiquement.

3.3 Rendu conditionnel : `v-if`, `v-else-if`, `v-else`, `v-show`

```
<script setup>
import { ref } from "vue";
const note = ref(12);
const visible = ref(true);
const connecte = ref(true);
</script>

<template>
  <!-- Les blocs se suivent : un seul est rendu (le premier dont la condition est
vraie) -->
  <p v-if="note >= 16">Très bien</p>
  <p v-else-if="note >= 10">Admis</p>
  <p v-else>Ajourné</p>
  <!-- → note = 12 : seul « <p>Admis</p> » existe dans la page -->

  <!-- v-show : l'élément reste toujours dans la page, seul son affichage change -
->
  <p v-show="visible">Affiché ou masqué</p>
  <!-- → visible = true : <p>Affiché ou masqué</p>
       → visible = false : <p style="display: none;">Affiché ou masqué</p> -->

  <!-- Regrouper plusieurs éléments sous une condition SANS balise supplémentaire
-->
  <template v-if="connecte">
    <h2>Bienvenue</h2>
    <button>Déconnexion</button>
  </template>
  <!-- → connecte = true : h2 + button sont dans la page, mais PAS de balise
<template> dans le DOM -->
</template>
```

	`v-if`	`v-show`
Mécanisme	**Ajoute / retire** l'élément du DOM	Laisse l'élément, applique `display: none`
Coût	Plus cher à basculer (création/destruction)	Plus cher à l'initialisation (tout est créé)
Usage	Condition rarement modifiée	Bascule fréquente (menu, onglet)
`v-else` possible ?	Oui	Non

3.4 Rendu de listes : `v-for`

`v-for` répète un élément **pour chaque élément d'une collection**.

```
<script setup>
import { ref } from "vue";
```

```
const taches = ref([
  { id: 1, texte: "Apprendre Vue", fait: false },
  { id: 2, texte: "Faire les exercices", fait: true },
  { id: 3, texte: "Réviser", fait: false }
]);
const objet = ref({ nom: "Ali", ville: "Tunis" });
</script>

<template>
  <!-- Sur un tableau : (élément, indice) -->
  <ul>
    <li v-for="(tache, index) in taches" :key="tache.id">
      {{ index + 1 }}. {{ tache.texte }}
    </li>
  </ul>
  <!-- → 1. Apprendre Vue
       → 2. Faire les exercices
       → 3. Réviser                    (l'indice commence à 0, d'où index + 1) --
>

  <!-- Sur un objet : (valeur, clé) -->
  <p v-for="(valeur, cle) in objet" :key="cle">{{ cle }} : {{ valeur }}</p>
  <!-- → nom : Ali
       → ville : Tunis -->

  <!-- Sur un nombre : va de 1 à n -->
  <span v-for="n in 5" :key="n">{{ n }} </span>
  <!-- → 1 2 3 4 5 -->
</template>
```

L'attribut `:key`

`:key` **est indispensable** : il donne à Vue un identifiant **unique et stable** pour chaque élément, afin de savoir lesquels ont été déplacés, ajoutés ou supprimés (plutôt que de tout recréer).

- ✅ Bon `key` : un identifiant propre à la donnée (`tache.id`).
- ❌ Mauvais `key` : l'`index` si la liste peut être triée, filtrée ou modifiée en milieu de liste (l'état des éléments se retrouve mélangé : voir l'exercice 2.5).

⚠ Ne combinez pas `v-for` et `v-if` sur le même élément

Voici le piège : vouloir n'afficher que les tâches **non terminées**.

```
<!-- ❌ À NE PAS FAIRE -->
<ul>
  <li v-for="tache in taches" v-if="!tache.fait" :key="tache.id">
    {{ tache.texte }}
  </li>
</ul>
<!-- → erreur à l'exécution : Cannot read properties of undefined (reading 'fait')
```

```
     + avertissement : « Property "tache" was accessed during render but is not
defined » -->
```

Pourquoi ça échoue ? En Vue 3, quand `v-if` et `v-for` sont sur le **même** élément, **`v-if` est évalué en premier** (il a une priorité plus haute). À ce moment, la variable `tache`, créée par `v-for`, **n'existe pas encore** : `!tache.fait` provoque donc une erreur. (En Vue 2, l'ordre était inversé : le code fonctionnait mais était inefficace. Les deux versions déconseillent ce mélange.)

Il y a **deux bonnes solutions**.

Solution A (recommandée) : filtrer la liste dans une propriété calculée, puis boucler sur la liste déjà filtrée.

```
<script setup>
import { ref, computed } from "vue";

const taches = ref([
  { id: 1, texte: "Apprendre Vue", fait: false },
  { id: 2, texte: "Faire les exercices", fait: true },
  { id: 3, texte: "Réviser", fait: false }
]);

// La liste filtrée est calculée UNE fois, mise en cache, et recalculée seulement
si taches change
const tachesAFaire = computed(() => taches.value.filter(t => !t.fait));
console.log(tachesAFaire.value);
// → [ { id: 1, texte: "Apprendre Vue", fait: false },
//     { id: 3, texte: "Réviser", fait: false } ]
</script>

<template>
  <ul>
    <!-- v-for ne voit que les tâches déjà filtrées : plus besoin de v-if -->
    <li v-for="tache in tachesAFaire" :key="tache.id">{{ tache.texte }}</li>
  </ul>
  <!-- → Apprendre Vue
       → Réviser                        (« Faire les exercices » n'apparaît pas) -
->
</template>
```

Avantages : le template reste **simple**, le filtrage est **mis en cache** (pas refait à chaque affichage), et on peut le **tester** et le **réutiliser** (par exemple `tachesAFaire.length` pour un compteur).

Solution B : une balise `<template v-for>` englobante, avec le `v-if` sur l'élément **à l'intérieur**.

```
<ul>
  <!-- La balise <template> porte le v-for (et la key) ; elle n'apparaît pas dans
le DOM -->
  <template v-for="tache in taches" :key="tache.id">
    <!-- Ici, « tache » existe déjà : le v-if peut l'utiliser -->
```

```
    <li v-if="!tache.fait">{{ tache.texte }}</li>
  </template>
</ul>
<!-- → Apprendre Vue
     → Réviser -->
```

Le `v-for` s'exécute **d'abord** (sur le `<template>`), puis le `v-if` de chaque `<li>` voit bien la variable `tache`. Cette solution est pratique quand la condition est **simple et locale**, ou quand il faut aussi afficher autre chose pour les éléments exclus (un `v-else`).

Cas où `v-if` + `v-for` ne posent pas de problème : quand la condition **ne dépend pas de l'élément** de la boucle. On place alors le `v-if` sur un élément **parent** :

```
<!-- ✅ La condition concerne la liste entière, pas chaque élément -->
<ul v-if="taches.length > 0">
  <li v-for="tache in taches" :key="tache.id">{{ tache.texte }}</li>
</ul>
<p v-else>Aucune tâche.</p>
<!-- → liste non vide : la <ul> avec ses <li>
     → liste vide : « Aucune tâche. » -->
```

Situation	Solution
Filtrer les éléments selon leurs propriétés	**A** : `computed` (recommandé)
Condition simple propre à chaque élément	**B** : `<template v-for>` + `v-if` à l'intérieur
Condition sur la liste entière	`v-if` sur l'élément parent

---

4. Propriétés calculées (`computed`)

Une **propriété calculée** est une valeur **dérivée** d'autres données réactives. Elle est **mise en cache** : recalculée **seulement** quand une des données dont elle dépend change.

```
<script setup>
import { ref, computed } from "vue";

const taches = ref([
  { id: 1, texte: "A", fait: true },
  { id: 2, texte: "B", fait: false }
]);

// computed reçoit une fonction qui RETOURNE la valeur
const restantes = computed(() => taches.value.filter(t => !t.fait));
console.log(restantes.value);          // → [ { id: 2, texte: "B", fait: false } ]

// Un computed peut en utiliser un autre
const message = computed(() => `${restantes.value.length} tâche(s) restante(s)`);
```

```
console.log(message.value);            // → "1 tâche(s) restante(s)"
</script>

<template>
  <p>{{ message }}</p>
  <!-- → 1 tâche(s) restante(s)      (se met à jour si une tâche est cochée ou
ajoutée) -->

  <li v-for="t in restantes" :key="t.id">{{ t.texte }}</li>
  <!-- → B -->
</template>
```

4.1 `computed` vs fonction

```
<script setup>
import { ref, computed } from "vue";

const taches = ref([{ id: 1, fait: false }, { id: 2, fait: true }]);

const restantes = computed(() => {
  console.log("computed exécuté");
  return taches.value.filter(t => !t.fait);
});

function calculerRestantes() {
  console.log("fonction exécutée");
  return taches.value.filter(t => !t.fait);
}
</script>

<template>
  <p>{{ restantes.length }}</p>
  <p>{{ restantes.length }}</p>
  <p>{{ restantes.length }}</p>
  <!-- → console : « computed exécuté » UNE seule fois (résultat mis en cache pour
les 3 usages) -->

  <p>{{ calculerRestantes().length }}</p>
  <p>{{ calculerRestantes().length }}</p>
  <p>{{ calculerRestantes().length }}</p>
  <!-- → console : « fonction exécutée » TROIS fois, puis à chaque nouvel
affichage du composant -->
</template>
```

Préférez `computed` pour toute valeur dérivée : même résultat, mais **moins de calculs**.

4.2 Règles d'or

- Un `computed` doit être **pur** : il **calcule et retourne**, sans effet de bord (pas de modification d'état, pas d'appel réseau, pas de `console.log` de production).

- Dans le script, on lit avec `.value` : `restantes.value`.
- Par défaut un `computed` est en **lecture seule**. Version modifiable (rare) avec `get` / `set` :

```
const prenom = ref("Ali");
const nom = ref("Ben Salah");

const nomComplet = computed({
  get: () => `${prenom.value} ${nom.value}`,        // lecture : assemble les deux
  set: (val) => {                                    // écriture : redécoupe la
valeur reçue
    const [p, ...reste] = val.split(" ");
    prenom.value = p;
    nom.value = reste.join(" ");
  }
});

console.log(nomComplet.value);          // → "Ali Ben Salah"
nomComplet.value = "Zeineb Trabelsi";     // appelle set()
console.log(prenom.value);              // → "Zeineb"
console.log(nom.value);                 // → "Trabelsi"
```

---

5. Watchers

Un **watcher** exécute un **effet de bord** en réaction à un changement : appel API, sauvegarde, journalisation. Contrairement à `computed`, il ne produit pas de valeur à afficher.

5.1 `watch`

```
import { ref, watch } from "vue";

const recherche = ref("");

// watch(source, callback) : le callback reçoit la nouvelle et l'ancienne valeur
watch(recherche, (nouvelle, ancienne) => {
  console.log(`"${ancienne}" → "${nouvelle}"`);
});

recherche.value = "vue";
// → "" → "vue"       (le callback n'est pas exécuté instantanément : Vue le lance
juste après
//                      le code en cours, au prochain cycle de mise à jour)
```

> **Les changements d'un même bloc de code sont regroupés.** Si on écrit `recherche.value = "vue"; recherche.value = "vue 3";` **à la suite**, dans la même fonction, le callback n'est appelé **qu'une fois**, avec le résultat final : `"" → "vue 3"`. Dans une vraie application, chaque frappe dans un champ est un événement distinct, donc chaque valeur est observée.

Observer **plusieurs sources** (tableau) :

```
const prenom = ref("Ali");
const nom = ref("Ben Salah");

watch([prenom, nom], ([p, n], [ancienP, ancienN]) => {
  console.log(`${ancienP} ${ancienN} → ${p} ${n}`);
});
prenom.value = "Zeineb";
// → Ali Ben Salah → Zeineb Ben Salah
```

Observer **une propriété d'un objet** : passer une **fonction** (un *getter*), car `watch(utilisateur.value.age, ...)` ne fonctionnerait pas (on passerait un simple nombre).

```
const utilisateur = ref({ nom: "Ali", age: 20 });

watch(() => utilisateur.value.age, (age) => {
  console.log("Nouvel âge :", age);
});
utilisateur.value.age++;
// → Nouvel âge : 21
```

Options

```
const taches = ref([]);

watch(taches, (valeur) => {
  localStorage.setItem("taches", JSON.stringify(valeur));
  console.log("Sauvegardé :", valeur.length, "tâche(s)");
}, { deep: true, immediate: true });
// → Sauvegardé : 0 tâche(s)     (immediate : exécuté tout de suite à la création
du watcher)

taches.value.push({ id: 1, texte: "A" });
// → Sauvegardé : 1 tâche(s)     (deep : un push à l'intérieur du tableau est
détecté)
```

Option	Effet
`deep: true`	Observe aussi les changements **à l'intérieur** (modification d'une propriété d'objet, `push` dans un tableau). Sans elle, seul le **remplacement** de `.value` est détecté pour un `ref`
`immediate: true`	Exécute le callback **dès la création** du watcher (avec l'ancienne valeur à `undefined`)
`once: true`	Ne se déclenche qu'**une seule fois**

5.2 `watchEffect`

`watchEffect` exécute la fonction **immédiatement** et **détecte automatiquement** les données réactives qu'elle lit : il se ré-exécutera quand l'une d'elles change. Pas besoin de désigner la source.

```
import { ref, computed, watchEffect } from "vue";

const taches = ref([{ id: 1, fait: false }, { id: 2, fait: false }]);
const restantes = computed(() => taches.value.filter(t => !t.fait).length);

watchEffect(() => {
  document.title = `${restantes.value} tâches`;     // lit « restantes » → devient
sa dépendance
  console.log("Titre mis à jour");
});
// → Titre mis à jour           (exécution immédiate ; l'onglet du navigateur
affiche « 2 tâches »)

taches.value[0].fait = true;
// → Titre mis à jour           (relancé car « restantes » a changé ; l'onglet
affiche « 1 tâches »)
```

	`watch`	`watchEffect`
Source	**Désignée** explicitement	**Détectée** automatiquement
Exécution initiale	Non (sauf `immediate`)	Oui, toujours
Ancienne valeur	Disponible	Non disponible
Usage	Réagir à **un** changement précis	Effet qui dépend de plusieurs données

5.3 `computed` ou `watch` ?

Besoin	**Outil**	**Exemple**
Calculer une valeur à **afficher**	`computed`	Prix TTC, liste filtrée, compteur
Déclencher une **action** (réseau, stockage, minuterie)	`watch` / `watchEffect`	Sauvegarde, requête API, titre de page

Règle simple : **« je veux une valeur »** → `computed` **; « je veux faire quelque chose quand ça change »** → `watch`**.**

---

6. Binding de classes et de styles

6.1 Classes

`:class` accepte un **objet** (clé = nom de classe, valeur = condition), un **tableau**, ou un mélange. Il **coexiste** avec l'attribut `class` statique.

```
<script setup>
import { ref } from "vue";

const actif = ref(true);
const erreur = ref(false);
const classeBase = "carte";
</script>

<template>
  <!-- Objet : la classe est présente si la valeur est vraie -->
  <div :class="{ actif: actif, 'text-danger': erreur }">...</div>
  <!-- → <div class="actif">...</div>     (actif = true ; erreur = false donc
text-danger absent) -->

  <!-- Tableau : toutes les classes listées sont appliquées -->
  <div :class="[classeBase, actif ? 'actif' : '']">...</div>
  <!-- → <div class="carte actif">...</div> -->

  <!-- Classe statique + dynamique : les deux sont fusionnées -->
  <div class="carte" :class="{ selectionnee: true }">...</div>
  <!-- → <div class="carte selectionnee">...</div> -->
</template>
```

6.2 Styles

```
<script setup>
import { ref } from "vue";
const couleur = ref("red");
const taille = ref(20);
const styleBase = { padding: "8px" };
const styleSurcharge = { color: "blue" };
</script>

<template>
  <!-- Objet : propriétés CSS en camelCase (fontSize) ou entre guillemets ('font-
size') -->
  <div :style="{ color: couleur, fontSize: taille + 'px' }">Texte</div>
  <!-- → <div style="color: red; font-size: 20px;">Texte</div> -->

  <!-- Tableau : plusieurs objets fusionnés, les derniers l'emportent -->
  <div :style="[styleBase, styleSurcharge]">Texte</div>
  <!-- → <div style="padding: 8px; color: blue;">Texte</div> -->
</template>
```

6.3 Exemple complet : liste de tâches en Vue

À comparer avec le mini-projet du chapitre 1 (JavaScript natif, avec `afficher()` à rappeler après chaque changement) :

```
<script setup>
import { ref, computed, watch } from "vue";

// --- ÉTAT ------------------------------------------------------------
const nouvelle = ref("");                                  // texte du champ de saisie
// Au démarrage : on relit les tâches sauvegardées (ou un tableau vide)
const taches = ref(JSON.parse(localStorage.getItem("taches") ?? "[]"));

// --- VALEUR DÉRIVÉE --------------------------------------------------
// Nombre de tâches non terminées : se met à jour toute seule
const restantes = computed(() => taches.value.filter(t => !t.fait).length);

// --- ACTIONS ---------------------------------------------------------
function ajouter() {
  const texte = nouvelle.value.trim();                 // supprime les espaces
autour
  if (!texte) return;                                  // ignore un texte vide
  taches.value.push({ id: Date.now(), texte, fait: false });
  nouvelle.value = "";                                 // vide le champ
}

function supprimer(id) {
  taches.value = taches.value.filter(t => t.id !== id);   // garde toutes sauf
celle-ci
}

// --- EFFET DE BORD ---------------------------------------------------
// À chaque modification de la liste (deep), on la sauvegarde dans le navigateur
watch(taches, v => localStorage.setItem("taches", JSON.stringify(v)), { deep: true
});
</script>

<template>
  <h1>Mes tâches</h1>

  <!-- v-model : champ lié à « nouvelle » (voir chapitre 5) ; Entrée ajoute la
tâche -->
  <input v-model="nouvelle" @keyup.enter="ajouter" placeholder="Nouvelle tâche">
  <button @click="ajouter">Ajouter</button>

  <ul>
    <!-- Un <li> par tâche ; la classe « fait » est ajoutée si la tâche est
terminée -->
    <li v-for="t in taches" :key="t.id" :class="{ fait: t.fait }" @click="t.fait =
!t.fait">
      {{ t.texte }}
      <!-- .stop empêche le clic sur × de déclencher aussi le clic du <li> (qui
cocherait la tâche) -->
      <button @click.stop="supprimer(t.id)">×</button>
    </li>
  </ul>
```

```
    <p>{{ restantes }} tâche(s) restante(s)</p>
  </template>

  <style scoped>
  .fait { text-decoration: line-through; color: gray; }
  </style>
```

Déroulement attendu :

Action de l'utilisateur	Résultat à l'écran
Taper « Réviser » puis `Entrée`	Une ligne « Réviser × » apparaît ; le champ se vide ; **1 tâche(s) restante(s)**
Ajouter « Courses »	Deux lignes ; **2 tâche(s) restante(s)**
Cliquer sur le texte « Réviser »	La ligne est barrée et grise ; **1 tâche(s) restante(s)**
Cliquer sur « × » de « Courses »	La ligne disparaît (sans cocher) ; **0 tâche(s) restante(s)**
Recharger la page (`F5`)	La liste est **conservée** (relue depuis `localStorage`)

Aucun appel `afficher()` n'est nécessaire : le DOM suit l'état. (`v-model` et les modificateurs d'événements comme `.stop` sont détaillés au chapitre 5.)

Résumé

- `{{ }}` affiche une **expression** ; les directives `v-bind` (`:`), `v-on` (`@`), `v-if`, `v-for` pilotent le DOM.
- `ref` est notre choix par défaut : `.value` dans le script, pas dans le template.
- `v-for` exige un `:key` unique et stable ; ne jamais mettre `v-if` sur le **même** élément que `v-for` (Vue 3 évalue `v-if` d'abord : la variable de boucle n'existe pas encore). Filtrer avec un `computed`, ou utiliser `<template v-for>`.
- `computed` : valeurs dérivées, pures et mises en cache. `watch` : effets de bord.
- `:class` et `:style` acceptent des objets et des tableaux.